Phase 10: Git, GitHub Actions CI, and Deployment

What we built

A CI/CD pipeline. On every push to GitHub, robots lint, type-check, build and test the app on a fresh Linux machine. If everything passes on `main`, they **deploy it live** to GitHub Pages:

```
git push ──> GitHub Actions
              ├─ check   lint · tsc + vite build · 71 Vitest tests + coverage   ─┐
(in parallel)
              └─ e2e     6 Playwright tests in real Chromium (desktop + phone)  ─┤
                                                                                  └─>
deploy → https://hamzawillcode.github.io/trading-dashboard/

(only if both pass, only on main)
```

File	Role
`.github/workflows/ci.yml`	The pipeline: triggers, three jobs, their steps
`app/vite.config.ts` → `base`	Serves the site from `/trading-dashboard/` on Pages (from `BASE_PATH`), and from `/` locally
`README.md`	CI status **badge** and the live link

Where to look on GitHub: the **Actions** tab (every run, its logs, failed-test annotations), the green ✓ / red ✗ next to each commit, and **Environments → github-pages** (deploy history and the live URL).

Concepts (theory only)

1. Git in one page (≈ version control you already know from backend work)

Command	What it does
`git status` / `git diff`	What changed since the last commit
`git add <files>`	Stage the changes for the next commit
`git commit -m "msg"`	Save a **snapshot** with a message. A commit = files + author + parent commit(s)
`git log --oneline`	History, newest first
`git switch -c feature/x`	Create a **branch** and move to it: a movable label on a line of commits
`git push` / `git pull`	Send your commits to GitHub (the **remote**, "origin") / fetch and merge theirs
`git merge` / `git rebase`	Combine branches: merge keeps both histories (merge commit), rebase replays your commits on top (linear)

The usual team flow (GitHub flow): branch off `main` → commit → push → open a **Pull Request** → CI runs on the PR → review → merge → CI deploys `main`. `main` is always deployable.

Good commits: small, one idea each, with a message saying *why*. Ours are one per phase, because each phase is one feature.

2. What CI is (Continuous Integration)

- Everyone merges into `main` **often**, and every change is **automatically built and tested** on a clean machine.
- It catches "works on my machine" problems: a missing file you never committed, a dependency only installed on your laptop, a type error in a file you didn't open.
- **The fast-feedback rule:** a red build within minutes, on the commit that broke it, is cheap to fix. A bug found weeks later is expensive.
- CI only works if you have **automated tests** (Phase 9). A robot can't click around.

3. CD: Continuous Delivery vs Continuous Deployment

- **Continuous Delivery:** every passing build *could* be released, and a human presses the button.
- **Continuous Deployment:** every passing build on `main` **is** released automatically. **Ours does this.**
- The tests are the **gate**: `deploy` has `needs: [check, e2e]`, so a failing test means nothing ships.

4. GitHub Actions vocabulary

Term	Meaning	In our file
Workflow	A YAML file in `.github/workflows/`	`ci.yml`
Trigger (`on:`)	The events that start it	`push` to main, `pull_request`, `workflow_dispatch` (manual button)
Job	A group of steps on **one fresh VM**; jobs run **in parallel** unless `needs:` links them	`check`, `e2e`, `deploy`
Runner	The machine	`ubuntu-latest`, created for the job and destroyed after
Step	One command (`run:`) or one reusable **action** (`uses:`)	`npm ci`, `actions/checkout@v4`
Action	A packaged, versioned step someone published	`checkout`, `setup-node`, `upload-artifact`, `deploy-pages`
Artifact	A file saved from a run that you can download	Playwright traces when a test fails; the built site for Pages
Secret / token	Credentials injected at runtime, never in the code	`GITHUB_TOKEN` (automatic). We need no other secrets
Environment	A deploy target with history and optional protection rules	`github-pages`

Jobs don't share files. Each one starts on an empty VM, which is why every job runs `checkout` and `npm ci` again. The **npm cache** (`cache: npm`, keyed on `package-lock.json`) makes the repeat installs fast.

5. Design decisions in our pipeline

- **`npm ci`, not `npm install`:** it installs exactly what the lock file says and fails if `package.json` and the lock disagree. **Reproducible builds.**
- **Fast checks and E2E run in parallel.** The total time is the slowest job, not the sum of all jobs.
- **Fail fast, cheapest first:** lint (seconds) → types/build → tests. The first red step stops the job.
- **`concurrency` + `cancel-in-progress`:** if you push twice, the old run is cancelled. Only the newest commit matters.
- **Least privilege:** the token is read-only by default (`permissions: contents: read`). Only `deploy` gets `pages: write` and `id-token: write`.
- **Deploy only from `main` pushes, never from PRs.** PR code hasn't been reviewed yet.
- **Retries only in CI** (`retries: 2`): a network blip on a shared runner shouldn't fail the build. Locally there are 0 retries, so flakiness stays visible.

- **Traces uploaded on failure:** download them from the run page and replay with `npx playwright show-trace`. You can debug a CI-only failure without reproducing it.
- **Pinned versions:** `node-version: 24`, `actions/checkout@v4`. A new major version can't silently change the build (for maximum safety, pin actions to a commit SHA).

6. Deploying a static site

`npm run build` turns our TypeScript + React into **plain files** in `app/dist/`: one `index.html`, one JS bundle and one CSS file with hashed names (`index-DMJoOVMd.js`). These are a **static site**: no server code, so any file host or CDN can serve them.

Host	How it works	Notes
GitHub Pages (ours)	The Actions workflow uploads `dist/`, and Pages serves it	Free for public repos, no extra account, lives under `/<repo>/`
Vercel / Netlify	Connect the GitHub repo. They build on every push and serve from a global CDN	Free tier, automatic **preview URL for every PR**, custom domains, root path
S3 + CloudFront	Upload files to a bucket behind a CDN	The AWS way, more setup

The `base` path gotcha: on Pages, the site lives at `…github.io/trading-dashboard/`, not at the root. If the HTML asked for `/assets/index.js`, the browser would look at `…github.io/assets/index.js` → 404 → **blank page**. Vite's `base` option prefixes every URL, giving `/trading-dashboard/assets/index.js`. Vercel/Netlify serve from the root, so they wouldn't need this.

Hashed filenames → caching: the hash changes whenever the file's content changes, so browsers can cache assets "forever". A new deploy has new names, and users get the new code. Only `index.html` must never be cached for long.

SPA routing note: we have no URL routes (one screen), so we skip the usual SPA trick: redirect all 404s to `index.html` so `/portfolio` doesn't 404 on refresh. Netlify does it with `_redirects`, Vercel with `rewrites`.

Environment variables at build time: Vite only exposes variables prefixed `VITE_` to the browser code, and they end up **in the public bundle**. Never put secrets in frontend env vars. We have none: Binance market data needs no key.

7. Where this runs in the "real world"

Same ideas, bigger scale: required status checks (can't merge a PR while CI is red), branch protection on `main`, preview deploys per PR, staging → production promotion, and rollbacks (re-deploy the previous build).

Try it yourself

1. Open the repo on GitHub → **Actions** → the latest “CI” run. Click each job and expand the steps to read the logs.
2. Click the **live link** in the README and open it on your phone (it works on mobile data: no laptop needed).
3. **Break the build on purpose:** on a branch, change a test expectation, push, and open a PR. Watch the red ✗ and the annotation on the exact line. Fix it, push again, and it goes green. (`git switch -c break-test` … `git push -u origin break-test`, then “Compare & pull request”.)
4. Find the run’s **Deployment** in the repo sidebar (Environments → github-pages) and see the history of deploys.
5. In the Actions tab, use **Run workflow** (`workflow_dispatch`) to re-run the pipeline without a new commit.

Resume mapping

> “**Git**” → the repo, one commit per phase, pushed to GitHub, and the branch → PR → merge flow “**GitHub Actions (CI/CD)**” → `.github/workflows/ci.yml`: lint, type-check, build, 71 unit/component tests and 6 Playwright E2E tests on every push and PR, then automatic deployment to GitHub Pages only when everything passes “an order flow with **Playwright on GitHub Actions CI**” → the `e2e` job (headless Chromium, desktop + Pixel 7, traces uploaded on failure)

Interview Qs

1. **What is CI and why use it?** Automatically building and testing every change on a clean machine. It catches integration bugs and “works on my machine” problems minutes after the commit, when they’re cheapest to fix.
2. **Continuous Delivery vs Continuous Deployment?** Delivery: always releasable, and a human releases. Deployment: every green build on main goes live automatically.
3. **Walk me through your pipeline.** On push/PR: two parallel jobs. One does lint → tsc + build → Vitest with coverage. The other runs Playwright in Chromium with a mocked Binance feed. If both pass on main, a third job builds with the Pages base path and deploys.
4. **Why `npm ci` in CI?** Exact, reproducible installs from the lock file. It’s faster, it fails on a lock mismatch, and it never rewrites the lock.
5. **How do you keep CI fast?** Parallel jobs, dependency caching, cheapest checks first, cancelling outdated runs, only installing Chromium, and keeping E2E few (the pyramid).
6. **A test passes locally but fails in CI. What do you do?** Read the logs, download the trace/artifacts, and check environment differences: OS, Node version, timezone, screen size, timing on a slower runner, missing env vars, test order and shared state.

7. **How do you handle secrets in GitHub Actions?** Repository/environment secrets, injected as env vars, masked in logs, and not given to PRs from forks. Use least-privilege `permissions`. Frontend bundles are public, so never ship secrets to the browser.
8. **What's a static site and how did you deploy it?** Pre-built HTML/JS/CSS with no server code. `vite build` produces `dist/`, and the workflow uploads it to GitHub Pages with `base` set to the repo path.
9. **Why are build filenames hashed?** Cache busting: assets can be cached forever because new content gets a new name. Only `index.html` stays fresh.
10. **Merge vs rebase?** Merge preserves the real history with a merge commit. Rebase rewrites your commits on top of the target for a linear history. Never rebase shared/pushed branches.
11. **How would you roll back a bad deploy?** Revert the commit (CI redeploys the old version), or re-run the previous successful deployment. Feature flags avoid needing a deploy at all.